clebran: Communication, Process, and World Models for ARC Program Synthesis

A combined summary of three focused papers — content-transfer falsification, process reorientation, and the world-modeling direction — on a small, fully-instrumented substrate at matched compute

Aidan Jones

July 2026

Source: clebran-arc-agi. This paper is a summary; the three focused papers are under papers/01-communication-cost, papers/02-how-you-learned-it, and papers/03-world-modeling. The long-form write-up is archived at papers/00-combined/paper-source/.

Abstract

`clebran` is a small, fully-instrumented substrate for testing hypotheses about agent design on ARC-AGI-1 program synthesis: typed programs evolved under pure outcome reward over Hodel's ~160 grid primitives, with a deterministic, exception-guarded interpreter, an audited candidate-evaluation budget, and per-task RNG substreams. We report three focused results on this substrate, all at matched compute.

Paper 1 — the falsification. A central hypothesis of emergent communication — that bandwidth-limited channels between divergent agents force compositionality and improve transfer — is *falsified* by its own strongest control. On a 30-task frontier subset at 4M candidate-evaluations per run, matched to 0.10% spread, a single agent that grows a central abstraction library (B5, a Stitch/DreamCoder-style learner) is the best arm; the only statistically significant comparison in the study is that it beats the recovered-primitive channel by **+0.85 evolution-phase solves (~2.2 standard errors)**. Lineages *do* diverge (pairwise primitive Jaccard 0.57 ± 0.10), but divergence is a *liability* for vocabulary quality: B5's mined library is more compositional (topographic similarity 0.33 vs 0.14) and more cross-task-reused (0.15 vs 0.05). The constraint regime is null across all four payload types, because adaptive operator selection self-regulates a failing channel regardless of bandwidth. *Communication is a transport mechanism, not a discovery mechanism.*

Paper 2 — the reorientation. Teaching *content* between agents is neutral (relevant) or harmful (irrelevant) at matched compute, and no amortisation across students rescues it. A controlled teach→test probe appeared to show a ~2× acquisition speedup from a relevant curriculum; adding the missing *no-teaching* baseline reverses the reading — relevant teaching is neutral (−986 evals, permutation p = 0.97) and irrelevant teaching is *slower* than not teaching, so the "2×" was the *harm* to a mis-targeted control, not a benefit of a relevant one. Positively, with *content held fixed* and only the *order* of acquisition varied, **order significantly changes capability** (forward beats reverse, paired McNemar p = 0.021; forward beats cold, p = 0.021): a dependency-violating order halves how much of the same curriculum is acquired (23% vs 47%). The effect is asymmetric — *don't teach backwards* matters more than teaching in exactly the right order, because an adaptive learner self-orders when the material is merely available. This is the study's first significant *process* effect against a wall of null *content* effects, and it reorients the question from what a system knows to how it learned it.

Paper 3 — a budget-conditional integration result. A *world-modeling* direction is built on the substrate: the clebran interpreter is a *perfect L1* world model, and a thin L2 planner on top (multi-pair, lookahead, loop-avoiding) picks the candidate primitive whose multi-pair predicted next-state is closest to the target. The trace dataset — 16,149 `(state, action, next_state)` rows from the 400 Hodel reference solvers, recorded by `interpreter.run_trace`

and shipped at `data/traces.jsonl` — is the labelled transition corpus. A k-NN L1 baseline is the wrong model class (0% out-of-distribution exact-match, 5% in-distribution, 99% coverage), establishing the floor for any learned L1. The **B5+planner arm** — B5 augmented with a planner-seeded individual in the initial population of every task — is *budget-conditional*: at 500K and 1M candidate-evaluation budgets on the 30-task frontier subset, 3 seeds, **B5+planner ties B5 at 10.00 vs 9.67 mean solves** with +0.33 evolution-phase solves and 1 unique task (f76d97a5) that B5 misses — the *only* positive comparison in the study, scoped to starved-GP budgets. At the headline 4M budget (5 seeds), B5 alone wins by 1.0 solve (13.4 ± 0.89 vs 12.4 ± 0.55), with the B5+planner's solved set a *strict subset* of B5's. The planner is exactly what ARC-AGI-3 was designed to test (arXiv:2603.24621, *Modeling* is one of four required capabilities), and the clebran typed-IR substrate and library-mining mechanism transfer directly.

The contribution. A falsification (paper 1), a reorientation (paper 2), and a substrate-integrated world-modeling direction that produces a budget-conditional integration result (paper 3: positive at starved-GP budgets, slight regression at the headline 4M budget). The asset of the work is the matched-compute measurement rig: a deterministic typed IR, a reproducible per-task RNG substream scheme, an audited candidate-evaluation budget, and a track record of reporting nulls. The substrate is ARC-AGI-1; the methodological lessons — always include a do-nothing control, report cost-to-solve alongside solve count, study the process axis not just the content axis, and build a perfect L1 oracle into the substrate as a world-model test bed — transfer to ARC-AGI-3 and to any other small-system agent-design question.

1. Introduction

Three experiments, all on a small, fully-instrumented substrate that the clebran rig provides. Each is a focused paper in its own right (and is published separately in this repository's `papers/` tree); this combined paper summarises them, points to the focused versions for full detail, and offers the synthesis: what the three together say about the design, the rig, and the question the project set out to ask.

The clebran design has three properties that make the substrate useful for these questions. (a) **Matched compute**: every arm is capped at the same total candidate-evaluations, metered inside the evaluator; wall-clock never enters a comparison. The audit is published (≤ 0.27% spread across multi-arm runs, 0.10% for the headline 20-seed frontier run). (b) **Determinism**: a typed-IR interpreter with a 399/400 reproduction gate on the Hodel reference solvers; per-task RNG substreams that make single-task debugging predict full-run behaviour. (c) **Honest baselines**: the controls include a single-population arm, an isolated-lineage ensemble, the central-library arm (B5) that embeds the same library mechanism the channel arms use, and (in paper 2) a *cold* no-teaching baseline that turns the teach→test verdict on its head.

The three results are complementary. Paper 1 falsifies the content-transfer hypothesis at the level of *what* an agent knows. Paper 2 turns the question to *how* a system learns, and shows that the order of acquisition is the only axis that moves capability with content held fixed. Paper 3 builds the substrate for a *next* question — whether a learned world model, used to plan, helps the search — and produces a **budget-conditional integration result**: positive at starved-GP budgets (500K-1M, ties B5 with 1 unique task) and slightly negative at the headline 4M budget (B5 alone wins by 1.0 solve). Read in order, the three together say: communication between divergent agents does not help; the process of learning matters, and is where the remaining purchase is; and the substrate's perfect L1 is the right place to start building a world-model direction.

2. The clebran substrate

A program is an SSA-style sequence of typed statements over Hodel's ~160 grid primitives, mirroring the shape of his hand-written solvers so those solvers parse losslessly into the IR (399/400 reproduce exactly through the deterministic, exception-guarded interpreter). Fitness is pure outcome reward — fraction of demonstration pairs solved exactly, tie-broken by a foreground/background-balanced per-cell accuracy scaled by an MDL length penalty. No supervised program targets; no neural component.

A lineage is a population evolving under typed mutation / crossover with elitism, a hall of fame, progress-weighted task scheduling, and adaptive operator selection (AOS) that weights each genetic operator by its recent success. Divergence is induced by distinct seeds and primitive-pool subsets. The library miner is a Stitch-style compression pass that antiunifies over constants and ranks candidate subroutines by compression utility.

The arm table:

arm	description
B0	single population, all compute
B1	N isolated lineages, no channel (union)
B2 / B3	constrained / unconstrained fragment channel
B4	constrained signal channel
B5	**single population + central abstraction library**
B6 / B7	constrained / unconstrained recovered-primitive channel
B8 / B9	constrained / unconstrained curriculum channel
B5+planner	**B5 + planner-seeded individual (paper 3)**

The three papers use this rig unchanged. The additions are: the trace dataset and the L1/L2 planner (paper 3); the teach→test protocol with a cold baseline (paper 2); and the matched-compute frontier ranking at 20 seeds × 4M evals (paper 1).

3. Paper 1 — On the Cost of Communication in Divergent Program Evolution

Headline. On a 30-task frontier at 4M candidate-evaluations per run, matched to 0.10% spread, **a single agent that grows a central abstraction library (B5) is the best arm**; the only statistically significant comparison in the study is that it beats the recovered-primitive channel by **+0.85 evolution-phase solves (~2.2 standard errors)**. The multi-agent-with-channel hypothesis is falsified by its own strongest control.

The ranking (20 seeds × 4M, evolution-phase solves /30).

arm	solves
B5 — single pop + central library	**2.65 ± 1.31**
B0 — single population	2.25 ± 1.07
B7 — recovered-primitive, unconstrained	1.85 ± 1.04
B6 — recovered-primitive, constrained	1.80 ± 1.06
B9 — curriculum, unconstrained	1.65 ± 0.75
B2 — fragment channel	1.60 ± 0.99
B8 — curriculum, constrained	1.60 ± 0.75
B1 — isolated	1.45 ± 0.94

Three forces explain the ranking. Splitting compute is expensive (three isolated lineages at 1.45 fall well short of one population at 2.25; the small populations converge prematurely). Communication pays back only part of that tax (recovered-primitive channels at ~1.8 beat isolation but never reach B0). The same library mechanism run centrally avoids the tax and adds to single-population search — B5 lands on top. The mechanism is real (lineages use received primitives, on average,

> *1 per solve) but the effect is small and underpowered: doubling seeds (8 → 20) shrank rather than confirmed B6 − B2 (+0.50 → +0.20), so it is real-but-small, not a near-miss on significance.*

Why divergence hurts vocabulary quality. B5's mined library is more compositional (topographic similarity 0.33 vs 0.14–0.15 for multi-agent arms) and more cross-task reused (0.15 vs 0.05). A single agent sees *every* task, so its mined abstractions generalise; a lineage that sees only the subset it solved mines task-idiomatic abstractions. Divergence — the premise of H1 — is precisely what *degrades* the shared vocabulary.

The constraint is null across all four payloads. H3 fails not only for fragments (the dose-response, where adaptive operator selection pins the fragment operator at 3% share regardless of bandwidth) but for recovered primitives (B6 ≈ B7, Δ = −0.05) and for curricula (B8 ≈ B9, Δ = −0.05). The same AOS self-regulation explains all three: unconstrained sharing cannot crowd out productive mutation because the learner down-weights a failing channel. In this substrate the constraint neither helps nor hurts.

What to keep, what to rethink. The library design — the compression miner, the MDL pressure, the Stitch-style antiunification — is the component of `clebran` worth keeping, and B5 is a strong "abstraction reuse without divergence" baseline any multi-agent framing must beat, not merely match. The multi-agent framing, the channel, and the constraint regime are the parts to rethink. The measurement rig is the asset.

See `papers/01-communication-cost/` for the full paper, the reproduction commands, and the frozen data.

4. Paper 2 — Not What You Know, How You Learned It

Headline. Teaching *content* between agents is neutral (relevant) or harmful (irrelevant) at matched compute, and no amortisation across students rescues it. Positively, *order* matters: with content held fixed, a curriculum taught foundations-first significantly outperforms the same curriculum taught foundations-last (paired McNemar p = 0.021), because a dependency-violating order halves how much of the same content is acquired (23% vs 47%). The effect is asymmetric — *don't teach backwards* matters more than teaching in exactly the right order, because an adaptive learner self-orders when the material is merely available.

The curriculum channel engages but does not transfer. In the full matched-compute ranking, B8 (constrained) and B9 (unconstrained) sit at the bottom (1.60 and 1.65 evolution-solves), with B8 − B1 = +0.15 (ns) and the constraint null (B8 ≈ B9). Yet the mechanism *works*: lineages received ~31 teaching tasks each and **solved 70–78% of them**, in prerequisite order, as designed. The students demonstrably learned the concepts their peers taught, and it bought essentially nothing on the real tasks. The bottleneck is not *acquiring* a taught concept — that worked — but that *a concept learned from demonstrations does not transfer to a new task that needs it*. This is the L2T transfer-vs-reconstruction gap at the level of whole concepts.

The teach→test probe needed a cold baseline. A controlled probe (taught vs control, 34 paired runs) appeared to show a ~2× speedup: among solves, the taught arm used 15% of its

budget versus 32% for control. Read alone, this says concepts transfer and buy speed. It is wrong, and the reason is instructive: the probe had no *no-teaching* baseline. It compared taught against a control that was itself *harmed* by being taught the wrong curriculum.

The correction: relevant teaching is neutral, irrelevant teaching harms. Adding a **cold** arm (no curriculum, full budget on the target) corrects the verdict.

arm	target-solve evals
cold	73,152
taught (relevant)	72,166
control (irrelevant)	116,067

paired difference	Δ	perm p	reads as
taught – cold	**−986**	**0.97**	relevant teaching ≈ no teaching
control – cold	+42,915	0.27	irrelevant teaching is *slower*
control – taught	+43,901	—	*what the probe measured*

The probe's 2× was the size of the harm to the mis-targeted control, mislabelled as a benefit. **Relevant concept teaching is neutral; irrelevant concept teaching causes negative transfer** — a taught vocabulary biases later search, so an agent that internalised the wrong abstractions searches *worse* than a naive one. Amortization cannot rescue it: per student, taught total L + T = 129k exceeds cold U = 73k, so the crossover `N > C/(U − L − T)` has no solution. Sharing a curriculum among many agents cannot pay off when the thing shared does not make an individual agent cheaper.

The positive result: order matters, at the extreme. The order experiment isolates *how*. It holds curriculum content fixed (a target's k prerequisite sub-tasks) and varies only the acquisition order, with identical task sets and identical budgets across arms; the arms differ only in the prerequisite score handed to the order-aware gate. 9 targets × 6 seeds → 216 runs.

order	curriculum acquired	target solve rate
forward (foundations first)	**47%**	**48.1%**
shuffled (unordered)	45%	38.9%
reverse (hardest first)	**23%**	33.3%
cold (no curriculum)	—	29.6%

comparison (paired McNemar)	wins	p
forward – reverse	9 vs 1	**0.021**
forward – cold	13 vs 3	**0.021**
forward – shuffled	7 vs 2	0.180
reverse – cold	5 vs 3	0.727

Teaching foundations-last lets the learner acquire only 23% of the same curriculum — half of forward's 47% — because a gate that demands the composition before its parts stalls. That halved acquisition significantly lowers capability. But fine-grained ordering is not the active ingredient: forward vs shuffled is not significant ($p = 0.18$), and shuffled acquires 45% ≈ forward's 47% — the learner self-orders when all steps are available. Strict gating matters mainly when it can do *harm* (reverse). As with teaching-vs-not, the effect is on *capability* (which targets solve at all), not the cost endpoint (all order-vs-order evals-to-solve contrasts are null). This is the study's first significant *process* effect and its first empirical purchase on the reorientation.

A capability audit. Studying process *richly* requires capabilities the substrate does not yet have: self-directed practice ("dreaming"), active choice of what to practise, retention without rigidity (plasticity), compositional generalisation, generalisation to genuinely unseen tasks, meta-learning, and reflection on how something was learned. The order result is a proof of concept that the *axis* is measurable; it also shows why measuring it richly needs a learner for which order does not self-correct as cheaply as pure GP's does.

See `papers/02-how-you-learned-it/` for the full paper, the reproduction commands, and the frozen data.

5. Paper 3 — A World-Model Substrate for ARC Program Synthesis

Headline. The clebran interpreter is a *perfect L1* world model for the clebran DSL. A thin L2 planner on top (multi-pair, lookahead, loop-avoiding) picks the candidate primitive whose multi-pair predicted next-state is closest to the target. The **B5+planner arm** — B5 augmented with a planner-seeded individual in the initial population of every task — is **budget-conditional** in its result. At 500K and 1M candidate-evaluation budgets on the 30-task frontier subset (3 seeds), B5+planner *ties* B5 at **10.00 vs 9.67** mean solves with +0.33 evolution-phase solves and 1 unique task (f76d97a5) that B5 misses — the *only* positive comparison in the study, scoped to starved-GP budgets. At the headline 4M budget (5 seeds), **B5 alone wins by 1.0 solve** (13.4 ± 0.89 vs 12.4 ± 0.55, ~1.8 SE on the per-seed delta), with the B5+planner's solved set a *strict subset* of B5's (2 unique B5 tasks: 0b148d64, f25fbde4). The planner seed is *useful*

when the GP is starved (500K-1M) and a slight *regression when the GP has enough budget to do the work itself* (4M).

The trace dataset. `data/traces.jsonl` is a JSONL file built by `scripts/build_traces.py`, which runs all 400 Hodel reference solvers through `interpreter.run_trace` and writes one row per trace step (state, action, next_state). **16,149 rows, 1,302 demo pairs, 6.3 MB.** The state-kind distribution is *tuple 43%, other 26%, grid 23%, int 7%*: most trace steps involve intermediate values (objects from `objects()`, indices from `ofcolor()`, etc.); only 23% are grid→grid transitions. The grid→grid subset (1,378 rows) is the cleanest test bed for the L1 predictor.

The L1 baseline (k-NN) is the wrong model class.

metric	value
KnnL1 OOD exact-match	0 / 353 (0.00%, coverage 36.83%)
KnnL1 OOD shape-match	27 / 353 (7.65%)
Identity OOD exact-match	19 / 353 (5.38%)
Identity OOD shape-match	189 / 353 (53.54%)
KnnL1 in-dist exact-match	14 / 280 (5.00%, coverage 99.29%)
KnnL1 in-dist shape-match	212 / 280 (75.71%)

The k-NN fails to *generalise* (0% OOD exact-match) and fails to *memorise* (5% in-distribution, 99% coverage). The `(state, action) → next_state` relationship is highly state-specific: the same primitive applied to two 1-cell-different states produces very different next-states, so Hamming-distance neighbours have inconsistent next-states and the majority vote loses. The k-NN is a *floor* — any learned L1 has to beat it to be useful.

The L2 planner. `clebran/world_model/l2_planner.py::L2Planner` is a thin layer on top of the clebran interpreter. For a given (state, target) pair, it iterates over a fixed candidate set — the most common grid→grid primitives in the Hodel reference solvers, with hand-defined default-argument strategies — and picks the primitive whose predicted next-state has minimum Hamming distance to the target. It is *multi-pair* (plans against all demo pairs simultaneously), *lookahead* (1-2 step rollout), and *loop-avoiding* (visited-state penalty).

metric	value
planner improves over identity (single-pair)	468 / 1,302 (35.9%)
Hodel first primitive in candidate set	241 (18.5%)
planner matches Hodel (when in candidate set)	68 / 241 (28.2%)
iterative planner (max 20 steps) solves (single-pair)	32 / 815 (3.9%)
iterative planner solves (multi-pair)	9 / 263 (3.4%)

The 3.4% multi-pair iterative solve rate is the *ceiling* any learned L1 has to recover, with the perfect L1 as the ground truth. The 28.2% Hodel match rate is the diagnostic: the planner is *not* picking the same primitive as the Hodel solver 72% of the time, because the Hodel solver's first move is not always the one with minimum Hamming distance to the target. The Hamming metric is too coarse for many ARC tasks.

The B5+planner arm.

arm	solved	evo-solved	test-exact	mean length
B5 (control, 500K)	9.67	2.67	9.67	1.61
B5+planner (L=1, 500K)	**10.00**	**3.00**	9.67	2.03
B5+planner (L=2, 500K)	**10.00**	**3.00**	9.67	1.70
B5+planner (L=1, 1M)	**10.00**	**3.00**	9.67	2.03

B5+planner ties B5 at the 10/30 frontier-subset ceiling, with +0.33 evolution-phase solves and 1 unique task B5 misses. The planner-seeded programs are slightly longer (2.0 vs 1.6 statements), which is the cost of carrying a non-trivial seeded individual. The integration works.

What flipped the result from "slight null" to "at parity". Two bugs in the first implementation made the planner plan toward the input grid instead of the target: (1) `target = input_grid` in `Lineage.seed_with_planner` (the first version discarded the demo's target and passed the input as both state and target, so the planner found a "no-action" minimum-distance primitive); (2) a narrow candidate set covering the top *10* rather than the top *20* grid→grid Hodel primitives. Fixing both — switching to the multi-pair API and expanding the candidate set — turned the slight null into parity.

At the headline 4M budget, B5 alone wins by 1.0 solve (5 seeds, 30-task frontier subset): B5 13.4 ± 0.89 vs B5+planner 12.4 ± 0.55 (~1.8 SE on the per-seed delta — directional but not yet 2 SE-bar significant at 5 seeds). The 5 per-seed deltas span -2 to +1, with 4/5 negative. The **B5+planner's solved set is a strict subset of B5's**: B5+planner has zero unique-to-it tasks, while B5 solves 2 unique tasks (0b148d64, f25fbde4) that B5+planner never solves in any of the 5 seeds. **The planner seed is a *budget- conditional intervention* — useful when the GP is starved (500K-1M, +0.33 evo, 1 unique task f76d97a5) and slightly negative**

when the GP has enough budget to do the work itself (4M, -1.0 solve, -2 unique tasks). A 20-seed 4M sweep is running to tighten the confidence interval; the headline pattern (B5 alone wins at 4M) is unlikely to reverse with 4x more seeds.

Two programmatic learned L1s (§3.2.5 of paper 3), built and compared head-to-head with the k-NN baseline and identity on the same held-out 20%-of-tasks split:

L1	exact%	shape%	mean_h*	pred/s	train_s
Identity	5.38	53.54	14.63	640K	0.00
KnnL1	0.00	7.65	20.56	106K	0.04
LookupTableL1	**25.50**	40.51	**4.20**	54K	0.01
GPL1	18.70	**52.69**	12.11	108K	9.40

The four L1s split into three roles: **Identity** is the floor (KnnL1 fails it on both axes — wrong model class); **LookupTableL1** is the *value-accurate* L1 (best exact-match and best Hamming, but 0% OOD hit rate — a true memoriser, not a learner); **GPL1** is the *shape-accurate* L1 (best shape-match, 108K pred/s, clebran's own GP evolves a small program per primitive in 9.4s total). The clebran GP cleanly separates the primitives into *learnable* (rot90, hmirror, vmirror, dmirror, cmirror — 100% test exact) and *not yet learnable* (fill, paint, replace — shape-only, because the trace dataset's `arg_kinds` field doesn't store the specific color constants). The next push is to extend the trace dataset to record arg *values* and expand the GP's program-length budget from 3 to 5-7.

Task-level evidence for the budget-conditional reading (extracted from the b5 / b5+planner logs by `scripts/inspect_task_programs.py`):

- `f76d97a5` — B5+planner unique at 500K-1M. The Hodel solution is `replace(I, 5, 4)` (replace centre color with 4) followed by a 4-shape cell-mask. The planner finds the right first statement in a 1-2 prefix; B5+planner's planner-seeded individual gives the GP a head start at starved budgets. At 4M, B5 alone finds similar programs (5-6 statements using mostcolor + switch + replace), and the unique-task advantage disappears.
- `0b148d64`, `f25fbde4` — B5 alone unique at 4M. Both are 19x21 / 9x9 large-grid tasks that need *compositional* primitives: `0b148d64` needs `subgrid(bordering(ofcolor, ofcolor, palette, last), I)`, `f25fbde4` needs `hconcat + mapply + upscale` (a tile program). The planner's grid→grid candidate set doesn't include these compositions, so the planner seed never has the right structure to bootstrap a solution. **The planner seed *blocks* the rare events B5 would otherwise find** — not by being actively wrong, but by *taking up a slot in the initial population*.

The next push. Four follow-ups are the most directly productive: (i) the **20-seed 4M-eval B5+planner frontier sweep** is currently running in the background (6 of 30 runs complete at the time of writing, ETA ~06:00) — it will turn the directional ~1.8 SE result at 5 seeds into a 2+

SE rejection (or fail to, which is also publishable); (ii) an object-aware distance metric that scores primitives on structural match rather than cell-wise Hamming, which would raise the planner's iterative solve rate; (iii) **re-arc-generated trace candidates** — re-arc's per-task generators would expand the planner's candidate set dramatically, addressing the planner-unfriendly tasks (`0b148d64` , `f25fbde4`); (iv) the **learned L1** is partially built — the GP-evolved L1 learns geometric primitives perfectly but is shape-only on value-computing primitives. Extending the trace dataset to record arg *values* (not just arg kinds) and expanding the GP's program-length budget from 3 to 5-7 are the natural next push. The clebran rig is the right test bed: the perfect L1 is the ground truth, the trace dataset is the labelled transition corpus, and the B5+planner arm is the L2 integration test.

ARC-AGI-3 is the natural long-term testbed. ARC-AGI-3 (arXiv:2603.24621) lists four required capabilities: *Exploration, Modeling, Goal-Setting, Planning and Execution*. The *Modeling* capability is "the ability to build a world model that represents how the environment works and can predict the results of unseen actions." The clebran L2 planner is exactly that: a model (the clebran interpreter) that predicts the results of unseen actions, used to plan. The clebran typed-IR substrate and library-mining mechanism transfer directly.

See `papers/03-world-modeling/` for the full paper, the reproduction commands, and the frozen data.

6. Synthesis

Read together, the three papers say the following.

Communication between divergent agents does not help at matched compute. The central-library arm (B5) is the best, and the only significant comparison in the study is that it beats the recovered-primitive channel. Divergence is real but *degrades* vocabulary quality. The constraint regime is null across all four payloads because adaptive operator selection self-regulates a failing channel. *Communication is a transport mechanism, not a discovery mechanism*. (Paper 1.)

The process of learning matters, and is where the remaining purchase is. Teaching *content* between agents is neutral or harmful at matched compute; no amortisation rescues it. But *order* matters: with content held fixed, a dependency-violating order halves how much of the same curriculum is acquired. The effect is asymmetric — *don't teach backwards* matters more than teaching in exactly the right order, because an adaptive learner self-orders when the material is merely available. (Paper 2.)

The substrate's perfect L1 is the right place to start building a world-model direction. The clebran interpreter is a perfect `(state, action) → next_state` oracle; a thin L2 planner on top uses it to pick candidate primitives; the B5+planner arm, integrating the planner as a seed for B5's central-library search, produces a **budget-conditional** result: positive at

starved-GP budgets (500K-1M, ties B5 at +0.33 evo and 1 unique task) and slightly negative at the headline 4M budget (B5 alone wins by 1.0 solve, the B5+planner's solved set a strict subset of B5's). The world-modeling direction is exactly what ARC-AGI-3 was designed to test, and the clebran typed-IR substrate and library-mining mechanism transfer directly. (Paper 3.)

Methodological lessons that recur across all three papers.

1. *Include a do-nothing control.* The teach→test probe's apparent 2× speedup was, against a cold baseline, the *harm* to a mis-targeted control. Without the cold arm, the paper would have reported a benefit that wasn't there.
2. *Report cost-to-solve alongside solve count.* The teaching content lives in cost (the speed effect, before correction) or in nothing (the corrected cold comparison); order lives in capability. A recurring lesson: report both, because different interventions surface on different endpoints.
3. *Study the process axis, not just the content axis.* The one axis that moves capability with content held fixed is *process* — the order in which knowledge is acquired. The substrate's asset is that, on the process axis, the instrument shows real structure where the content axis showed only nulls.
4. *Build a perfect L1 oracle into the substrate as a world-model test bed.* The clebran interpreter is exactly that: a deterministic, exception-guarded, 399/400-validated `(state, action) → next_state` function. A learned L1 can be measured against its predictions, and a planner using the interpreter as L1 establishes the ceiling any learned L1 has to recover.

The asset of the work is the matched-compute measurement rig. A cheap critique of any null in evolutionary synthesis is "you under-powered the channel." The power check doubled seeds and raised budget; the effect shrank and the standard error narrowed, so the result is real-but-small, not a near-miss. The rig — audited candidate-evaluation budgets, per-task RNG substreams that make single-task debugging predict full-run behaviour, a deterministic interpreter with a 399/400 reproduction gate — is what makes the nulls trustworthy. The substrate is ARC-AGI-1; the methodological lessons transfer to ARC-AGI-3 and to any other small-system agent-design question.

7. Open work

The three papers together open four productive work items, in order of immediacy.

1. **20-seed 4M-eval B5+planner frontier sweep.** Match the headline B5 result of paper 1. The per-seed 1M pilot of paper 3 is at the 10/30 frontier-subset ceiling in all 3 seeds, so the sweep is a *consistency* test across more seeds, not a magnitude test. The 20-seed 4M-eval configuration is the standard match-compute audit.

2. **Object-aware distance metric.** The L2 planner's Hamming metric is too coarse for many ARC tasks — the 28.2% Hodel match rate is the diagnostic. An object-aware distance that scores primitives on structural match (counting matching objects by color and shape) would raise the planner's iterative solve rate. This is the natural improvement to the planner.
3. **The learned L1.** A small neural or programmatic approximation of the interpreter that, once it generalises, replaces the perfect-but-slow interpreter as the planner's rollout engine. The trace dataset (16K rows) is the labelled transition corpus; the perfect L1 is the ground truth. The k-NN baseline (0% OOD / 5% in-dist) is the floor to beat.
4. **Re-arc generated trace candidates + ARCTraj integration + ARC-AGI-3 evaluation.** The §9f build order in `roadmap.md` lists the long-horizon items: re-arc-generated trace candidates (to expand the candidate set with primitives inferred from per-task generators); ARCTraj integration (the 10K human step-by-step ARC trajectories as a cross-modality signal); and ARC-AGI-3 as the long-term testbed, where *Modeling* is one of four required capabilities.

References

See each focused paper's `references.md` for the full bibliography. The most-cited works across the three papers:

- Ha & Schmidhuber (2018) *World Models* — the L1/L2/L3 framing for planning in a learned simulator.
- DreamerV2 / DreamerV3 (Hafner et al. 2019–2023) — the data-efficient world-model + planning line.
- Ye et al. (2026) *Agentic World Modeling* (arXiv:2604.22748) — the L1 Predictor / L2 Simulator / L3 Evolver taxonomy.
- ARCTraj (Kim et al. 2026, arXiv:2511.11079) — 10K human step-by-step ARC trajectories.
- ARC-AGI-3 (arXiv:2603.24621) — the interactive turn-based benchmark with Modeling as a required capability.
- Hodel's *re-arc* — per-task generators for unlimited synthetic ARC trace data.
- Stitch (Ellis et al. 2018), DreamCoder (Ellis et al. 2021) — the compression-miner / central-library mechanism the B5 arm is built on.
- L2T / NEO (Baek et al. 2026, ICML 2026 oral) — the transfer-vs-reconstruction framing the paper 2 teach→test probe inherits.
- CompressARC (Liao & Gu 2025) — the direct neural-ARC competitor on the "intelligence as compression" axis.
- Chollet's *On the Measure of Intelligence* — the skill-acquisition-efficiency framing the paper 2 reorientation inherits.

Per-paper pointers

paper	directory	full paper
1. Communication cost	`../01-communication-cost/`	`paper.md`
2. How you learned it	`../02-how-you-learned-it/`	`paper.md`
3. World modeling	`../03-world-modeling/`	`paper.md`
Long-form archive	`paper-source/`	the 53-page write-up this combined paper summarises